

Java Lab Record: Collection Framework - Letter Combinations of a Phone Number

Objective / Aim

To design and implement a Java program that, for a given string of digits from 2-9, returns **all possible letter combinations** the number could represent on a telephone keypad, using the **Collection Framework** interfaces: **List**, **Set** and **Queue** (with **Map** for the keypad).

Problem Statement

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. A mapping of digit to letters (just like on the telephone buttons) is used. Note that 1 does not map to any letters.

2 -> abc	3 -> def	4 -> ghi	5 -> jkl
6 -> mno	7 -> pqrs	8 -> tuv	9 -> wxyz

Expected Output

Input: "23"

Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

Algorithm / Logic Description

1. Create a class named PhoneLetterCombinations.
2. Store the keypad in a Map: digit -> letters (2=abc ... 9=wxyz).
3. Create a method letterCombinations(digits) that returns a List<String>.
4. If the input is null or empty, return an empty list.
5. Use a Set (the keys of the Map) to check that every digit is 2-9.
If not, throw IllegalArgumentException.
6. Create a Queue and add one empty string "" to it.
7. For each digit in the input:
 - a. Note the current size of the queue.
 - b. Repeat that many times: remove one string (prefix) from the front, and add prefix + each letter of the digit to the back.
8. When all digits are processed, the queue holds every combination.
9. Copy the queue into the result List and return it.
10. In main(), call the method for sample inputs and print the results.

Real-time Application

Predictive text on old mobile keypads (T9), phone-number to word mnemonics (like 1-800-FLOWERS), PIN / password guessing tools used by security testers, and any problem that needs all combinations of choices.

PhoneLetterCombinations.java

Source Code

```
package beyond;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;

public class PhoneLetterCombinations {
    // Map: digit -> letters, exactly like the buttons of a telephone.
    private static final Map<Character, String> KEYPAD = new HashMap<>();

    static {
        KEYPAD.put('2', "abc");
        KEYPAD.put('3', "def");
        KEYPAD.put('4', "ghi");
        KEYPAD.put('5', "jkl");
        KEYPAD.put('6', "mno");
        KEYPAD.put('7', "pqrs");
        KEYPAD.put('8', "tuv");
        KEYPAD.put('9', "wxyz");
    }
}
```

```

    public static List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();           // List: final
        answer, keeps order
        if (digits == null || digits.isEmpty()) {
            return result;
        }

        Set<Character> validDigits = KEYPAD.keySet();     // Set: the digits 2-
        9 that are allowed
        for (char digit : digits.toCharArray()) {
            if (!validDigits.contains(digit)) {
                throw new IllegalArgumentException("Only digits 2-9 are
allowed: " + digit);
            }
        }

        Queue<String> queue = new LinkedList<>();         // Queue:
        combinations built so far (FIFO)
        queue.add("");                                   // start with one
        empty combination

        for (char digit : digits.toCharArray()) {
            int size = queue.size();                       // only process
            combinations of the previous level
            for (int i = 0; i < size; i++) {
                String prefix = queue.remove();           // take the oldest
                combination from the front
                for (char letter : KEYPAD.get(digit).toCharArray()) {
                    queue.add(prefix + letter);          // add it back with
                }
            }
            one more letter

            result.addAll(queue);                          // copy the finished
            combinations into the list
            return result;
        }

        private static void show(String digits) {
            List<String> output = letterCombinations(digits);
            StringBuilder sb = new StringBuilder("[");
            for (int i = 0; i < output.size(); i++) {
                sb.append("\").append(output.get(i)).append("\");
                if (i < output.size() - 1) {
                    sb.append(", ");
                }
            }
            sb.append("]");
            System.out.println("Input : \"\" + digits + "\"");
            System.out.println("Output: \" + sb);
            System.out.println("Total combinations: \" + output.size());
        }

        public static void main(String[] args) {
            show("23");

```

```

        System.out.println("-----");
        show("9");
        System.out.println("-----");
        show("");
    }
}

```

1. Concept and Exception in Simple Words

Collection Framework concepts used

Collection -> the root interface for groups of objects in Java.
List -> an ORDERED collection that allows duplicates.
ArrayList is used for the final answer, so combinations stay in the order ad, ae, af, bd ...
Set -> a collection with NO duplicates. Here the keys of the Map (2-9) act as a Set of valid digits: contains() checks quickly whether a digit is allowed.
Queue -> FIFO (First In, First Out). LinkedList is used as a Queue: add() puts at the back, remove() takes from the front.
Map -> stores key -> value pairs (digit -> letters). It is part of the Collection Framework but is not a Collection itself.

Why a Queue?

The queue works like a waiting line. We take the oldest partial word from the front, add every possible next letter, and put the new words at the back. So words grow one letter at a time, level by level, and the output order is automatically correct.

```

Digits "23"
Start      : [""]
After digit 2 : ["a", "b", "c"]
After digit 3 : ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

```

Exception in simple words

An exception is an unexpected problem that stops the normal flow of a program.

Exception	Kind	Where it comes from
IllegalArgumentException	Unchecked	Thrown by this program on purpose if the input has a character that is not 2-9 (for example "1" or "a"). It tells the caller the input is wrong
NullPointerException	Unchecked	Would happen if digits were null and we used it. The program avoids this by checking digits == null first

No try-catch is needed, because the sample inputs in main() are valid.

2. Step-wise Execution (Input "23")

1. `main()` calls `show("23")`, which calls `letterCombinations("23")`.
2. `result = []` (empty List). Input is not empty, so we continue.
3. Validation: '2' and '3' are both in `validDigits` -> OK.
4. `queue = []`
5. Digit '2' (letters "abc"), size = 1:
 remove "" -> add "a", "b", "c"
 `queue = [a, b, c]`
6. Digit '3' (letters "def"), size = 3:
 remove "a" -> add "ad", "ae", "af"
 remove "b" -> add "bd", "be", "bf"
 remove "c" -> add "cd", "ce", "cf"
 `queue = [ad, ae, af, bd, be, bf, cd, ce, cf]`
7. `result.addAll(queue)` -> result has 9 combinations.
8. `show()` prints the List in the format `["ad", "ae", ...]` and prints "Total combinations: 9".
9. `main()` prints a separator and repeats for "9" and "".

Count of combinations

"23" -> $3 \times 3 = 9$
"9" -> 4 letters = 4
"" -> 0 (empty input gives an empty list)

3. Algorithm

START

1. Fill Map `KEYPAD` with digit -> letters
2. CALL `show("23")`, `show("9")`, `show("")`

STOP

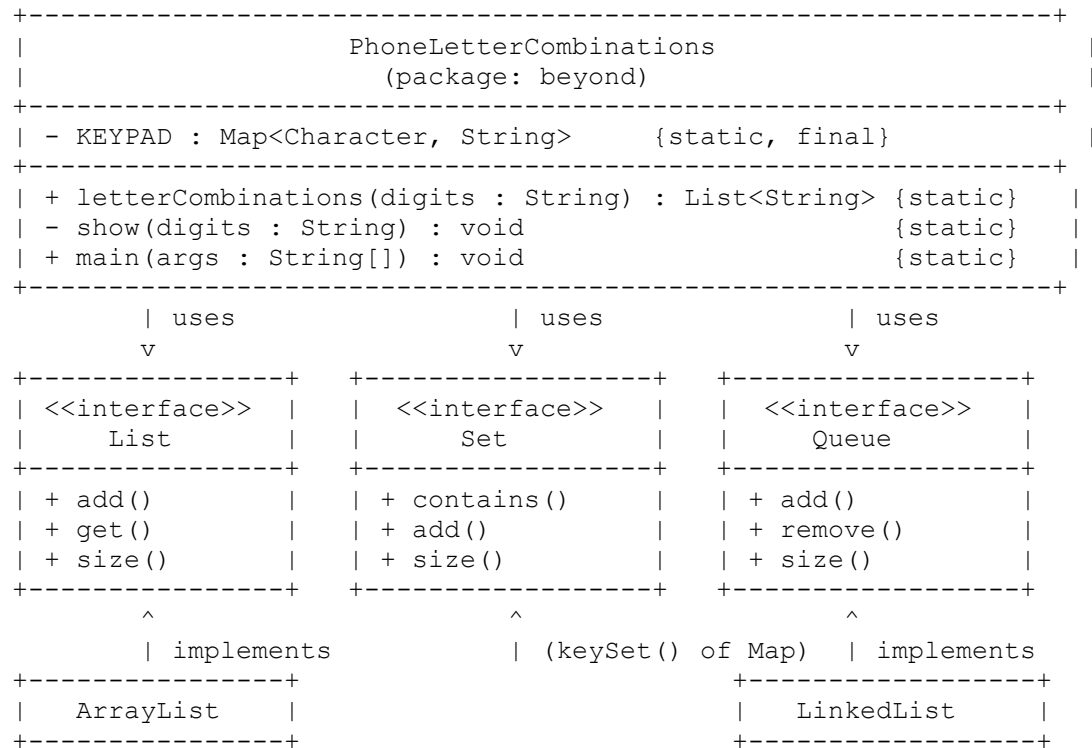
`letterCombinations(digits)`

1. `result = new empty List`
2. IF `digits` is null OR empty: RETURN `result`
3. FOR each digit in `digits`:
 IF digit is not a key of `KEYPAD` (Set check):
 THROW `IllegalArgumentException`
4. `queue = new Queue; queue.add("")`
5. FOR each digit in `digits`:
 `size = queue.size()`
 REPEAT size times:
 `prefix = queue.remove()`
 FOR each letter in `KEYPAD.get(digit)`:
 `queue.add(prefix + letter)`
6. `result.addAll(queue)`
7. RETURN `result`

`show(digits)`

1. `output = letterCombinations(digits)`
2. Build the text `["x", "y", ...]` from the List
3. PRINT Input, Output and Total combinations

4. UML Class Diagram



Map<Character,String> KEYPAD is implemented by HashMap.

Legend: - private, + public, {static} belongs to the class itself.

Compilation and Execution

Folder structure:

```
project/
|-- beyond/
    |-- PhoneLetterCombinations.java
```

Compile:

```
javac -d out beyond/PhoneLetterCombinations.java
```

Run:

```
java -cp out beyond.PhoneLetterCombinations
```

📌 Sample Output

Input : "23"

Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

Total combinations: 9

Input : "9"

Output: ["w", "x", "y", "z"]

Total combinations: 4

Input : ""

Output: []

Total combinations: 0

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: [ad, ae, af, bd, be, bf, cd, ce, cf]
Please enter digits from 2 to 9 only.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: 23
[ad, ae, af, bd, be, bf, cd, ce, cf]
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: 2
[a, b, c]
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: 5
[j, k, l]
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: 56
[jm, jn, jo, km, kn, ko, lm, ln, lo]
PS C:\Users\Admin\OneDrive\Documents\basicjava> c::; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.LetterCombinations'
Enter digits from 2 to 9: 100
Please enter digits from 2 to 9 only.
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

✦ Result

The Letter Combinations of a Phone Number program was successfully implemented in Java using the Collection Framework. A `Map` stored the keypad, a `Set` validated the digits, a `Queue` built the combinations level by level, and a `List` returned the final answer. For the input "23" the program produced all 9 combinations: ad, ae, af, bd, be, bf, cd, ce, cf, matching the expected output.